

SYSTEM S0 — SOFTWARE DESIGN DOCUMENT

The shared contract every team builds against: gateway & auth, the cross-system event and call catalog, RBAC, the financial posting map, and platform-wide engineering conventions

CONSOLIDATES FORMER MODULES

M1 Identity & Access

M2 Human Resources

M3 Payroll

M4 Leave & Attendance

M5 Inventory & Purchasing

M6 Restaurant & F&B

M7 Hotel Room Management

M8 Finance & Accounting

M9 Reporting & Analytics

DOCUMENT INFORMATION

Document type	Software Design Document — Developer Edition
Intended audience	All four development teams — read this first, before your system-specific SDD
Version	1.0
Date	2026-06-15
Companion documents	S1 Identity & Access · S2 Workforce & Payroll · S3 Hospitality Operations · S4 Finance & BI

Table of Contents

1. From Nine Modules to Four Systems	3
2. New Integration Decisions (ID1–ID12)	5
3. Gateway & Routing	7
4. Authentication & Authorization	8
4.1 JWT Flow (S1 issues, S2–S4 verify)	8
4.2 JWT Payload (updated — adds <code>name</code> , ID2)	8
4.3 Service-to-Service Auth	8
4.4 JWKS & Key Rotation	9
5. Cross-System Integration Map	10
5.1 Event Bus Topology	10
5.2 Cross-System Event Catalog	10
5.3 Cross-System Synchronous Calls	12
5.4 S4's Read-Through Cache (unchanged pattern, fewer upstreams)	13
5.5 What Became Internal (the actual simplification)	13
6. Internal Module Maps (summary — full detail in each system's SDD)	15
7. RBAC — Roles, Systems, and the Permission Convention	16
7.1 Permission Naming Convention (ID9)	16
7.2 Role → System Access Matrix (12 roles, unchanged set from the macro design)	16
8. Financial Posting Map (S2/S3 → S4)	17
9. Platform Conventions	19
10. Deployment Topology	22
11. Decision Register (Full Index)	24
12. Glossary & Legacy Cross-Reference	25

NOTE

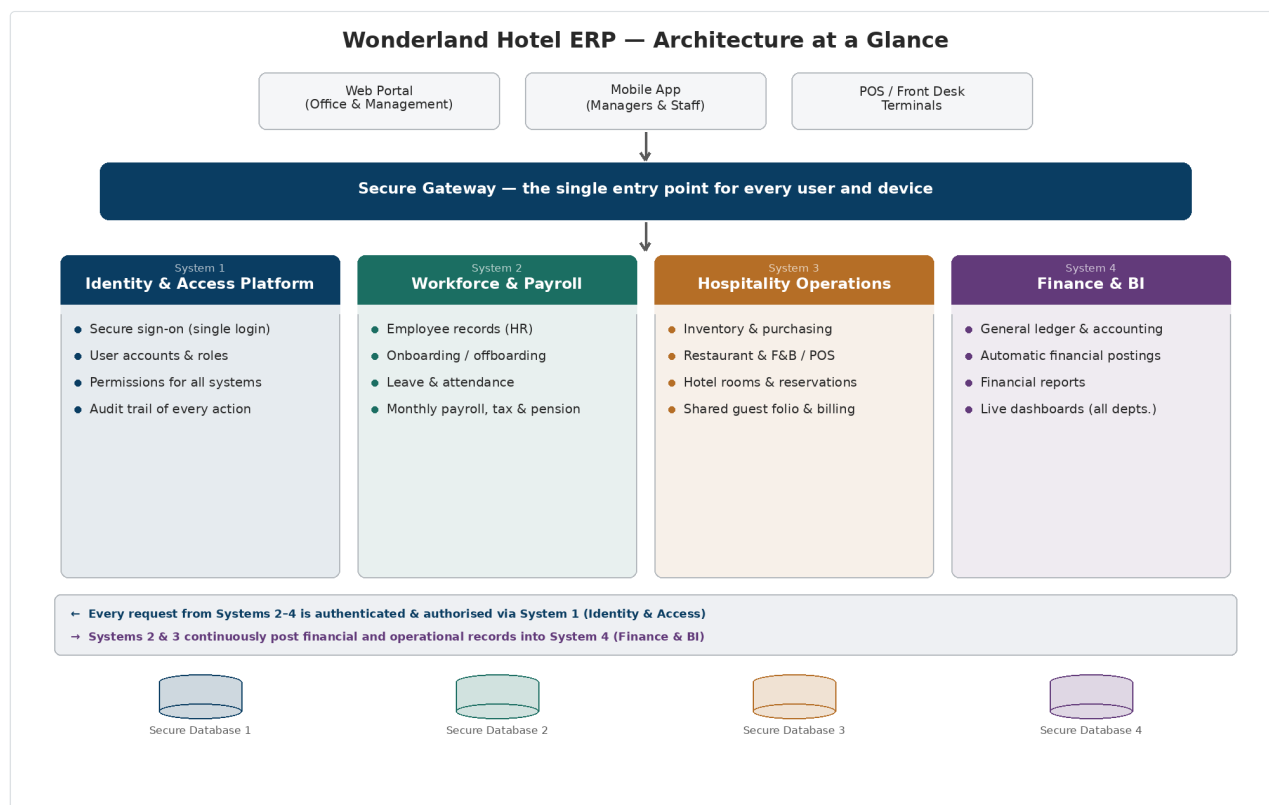
How to use this document. Each of the four developer teams (S1, S2, S3, S4) should read this document **first**, before their own system SDD. It defines everything that crosses a system boundary — APIs, events, roles, ledger postings, and shared engineering conventions — so that each system can be built independently against a fixed contract. Anything **not** in this document (internal data models, internal workflows, internal module structure) is the responsibility of the relevant system's own SDD and is no one else's concern.

Companion documents: [S1 — Identity & Access Platform](#) · [S2 — Workforce & Payroll](#) · [S3 — Hospitality Operations](#) · [S4 — Finance & Business Intelligence](#) Derived from: [Wonderland Hotel ERP Macro Architecture SDD.pdf](#) (business framing, four-system grouping) and the resolved engineering baseline in [00_MASTER_Wonderland_ERP_Design_Documentation.md](#) / Appendices A–D (the original 9-module design, now being consolidated).

1. From Nine Modules to Four Systems

The original design specified **nine independent Laravel microservices** (M1–M9), each with its own database, talking to each other over HTTP with a shared service-key and a Redis event bus. The macro architecture document regroups these into **four systems**, each of which is now a **modular monolith**: one Laravel application, one MySQL database, with the former modules living on as **internal modules** inside that application.

System	Internal Modules (former)	Database	Container	Internal Port
S1 — Identity & Access Platform	Identity & Access (M1)	wh_s1_db	s1-identity	9001
S2 — Workforce & Payroll	HR (M2) · Payroll (M3) · Leave & Attendance (M4)	wh_s2_db	s2-workforce	9002
S3 — Hospitality Operations	Inventory & Purchasing (M5) · Restaurant & F&B (M6) · Hotel & Room Mgmt (M7)	wh_s3_db	s3-hospitality	9003
S4 — Finance & Business Intelligence	GL & Accounting (M8) · Reporting & Analytics (M9)	wh_s4_db	s4-finance-bi	9004



Wonderland Hotel ERP architecture at a glance. A single secure gateway fronts all client traffic and routes it to one of four independently-deployable systems, each a self-contained modular monolith with its own database.

What this changes, mechanically:

- Where the old design had a former module call a *sibling* module that is now **inside the same system**, that call becomes an **in-process method call** (a service class, an Eloquent relationship, or a single DB transaction) — no HTTP, no JWT, no `X-Service-Key`, no event bus.
- Where the old design had a module call a module that is now in a **different system**, that call **remains cross-system** — it keeps using the gateway-fronted API + JWT (for user-driven calls) or the internal service-to-service pattern (for system-to-system calls), exactly as before, just renumbered S1–S4.
- The **Redis event bus** survives, but only for genuinely cross-system events. Several of the original 13 event-bus channels collapse into in-process Eloquent observers (Section 5.3).

Section 2 walks through every cross-cutting decision this collapse required. Sections 3–10 then specify the resulting (smaller, cleaner) cross-system contract in full.

2. New Integration Decisions (ID1–ID12)

These are **new design decisions** made specifically for the four-system consolidation — they did not exist in the original 9-module design. Each is referenced by ID throughout this document and the four system SDDs. A full index is repeated in Section 11 for traceability.

ID	Decision	System(s) Affected
ID1	New: <code>EmployeeArchived</code> now triggers <code>DeactivateLinkedUserAccount</code> in S1 — closing a real gap in the original design where an offboarded employee's login remained active indefinitely.	S2 → S1
ID2	New: JWT payload gains a <code>name</code> claim (employee's full display name). S1's <code>users</code> table gains <code>display_name</code> , populated from S2's employee-lifecycle events. This removes the need for S3 to call back to S2 just to resolve a cashier's or employee's display name.	S1, S2, S3
ID3	Simplification: the old “wash account” GL-netting workaround for hotel-guest F&B charges (previously needed because M6 and M7 were separate services posting to M8 independently) is no longer needed — M6 and M7 are now both inside S3, so S3 nets hotel-guest F&B revenue against the room folio internally and sends one clean journal to S4.	S3 → S4
ID4	Simplification: ingredient dispatch on order finalization (was <code>M6 → M5 POST /items/dispatch</code>) is now a single in-process DB transaction inside S3 — the Restaurant module calls the Inventory module's <code>FifoFefoService::dispatch()</code> directly, in the same transaction as the order/bill update.	S3 internal
ID5	Simplification: resolving a room's active folio and posting an F&B charge to it (was <code>M6 → M7 GET /active-folio → POST /fnb-charge</code> , Decision B4) is now a single in-process call to the Hotel module's <code>FolioService::postChargeForRoom()</code> .	S3 internal
ID6	Simplification: all of Payroll's former cross-service pulls from HR and Leave (<code>GET /employees</code> , pension category, unused-leave-balance,	S2 internal

ID	Decision	System(s) Affected
	overtime-approved) become direct Eloquent queries/relationships inside S2. Payroll always reads current data — the old “pull-based, re-read at run start” framing is no longer needed because there is no staleness to manage.	
ID7	Simplification: <code>JournalPosted</code> → dashboard cache invalidation (was <code>M8 → M9</code> over the bus) is now an in-process Eloquent model observer inside S4.	S4 internal
ID8	Bus topology: cross-system event channels are renamed <code>wh.events.{system}.{entity}.{action}</code> (was <code>{module}</code>). 3 systems emit (S1, S2, S3); S4 emits none — it is purely a GL-of-record + aggregator. Each system runs one bus-subscriber worker (down from up to 3 per module previously). See Section 5.	All
ID9	Permission naming convention updated to <code>S{n}.{domain}.{resource}.{action}</code> (e.g. <code>S2.payroll.runs.execute</code> , <code>S3.hotel.folios.read</code>). Each system SDD owns its own permission catalog; this document owns only the role → system access matrix (Section 7).	All
ID10	Infrastructure reduction: <code>event_outbox</code> now needed in only S1, S2, S3 (3, was 8); <code>idempotency_keys</code> now needed in S2, S3, S4 (3, was 5).	S1, S2, S3, S4
ID11	Gateway reduction: 4 upstreams (<code>/s1/ ... /s4/</code>), down from 9.	Gateway
ID12	Retained cross-system call: the food-consumption deduction push (<code>S3 → S2 POST /employees/{id}/deductions</code> , was <code>M6 → M3</code>) is the one business-data synchronous call that remains between S2 and S3 (besides auth/directory). It stays synchronous and idempotent (financial reconciliation needs an immediate confirmation, not eventual consistency).	S3 → S2

3. Gateway & Routing

A single reverse-proxy container, `wh-gateway` (Nginx), is the only externally-published entry point (ports 80/443 → TLS via Let's Encrypt/certbot). S1–S4 are reachable only on the internal `wh-internal` Docker network.

```
Internet —TLS—> wh-gateway (Nginx, 80/443)
├ /s1/api/v1/* → s1-identity:9001/api/v1/*
├ /s2/api/v1/* → s2-workforce:9002/api/v1/*
├ /s3/api/v1/* → s3-hospitality:9003/api/v1/*
└ /s4/api/v1/* → s4-finance-bi:9004/api/v1/*
```

Routing convention: external clients call `https://api.wonderlandhotel.com/s{n}/api/v1/...`; the gateway strips the `/s{n}` prefix and proxies to `s{n}-service:900{n}/api/v1/...`. Example block (repeated for S1–S4):

```
location /s2/ {
    rewrite ^/s2/(.*)$ /$1 break;
    proxy_pass http://s2-workforce:9002;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-Proto $scheme;
    proxy_set_header X-Request-Id $request_id;
}
```

Health checks: each system exposes `GET /api/v1/health` → `{"status":"ok","system":"S2","version":"1.0"}` (unauthenticated), used by Docker Compose healthchecks and the gateway's upstream checks (`503` returned for a down system rather than hanging).

Rate limiting (applied at the gateway):

Route pattern	Limit	Rationale
<code>/s1/api/v1/auth/login</code>	10 req/min per IP	Brute-force mitigation, complements S1's 5-attempt account lockout
<code>/s1/api/v1/auth/refresh</code>	30 req/min per IP	Normal token-refresh traffic
All other <code>/s{n}/api/v1/*</code>	300 req/min per authenticated user (keyed on JWT <code>sub</code> via an Nginx/njs snippet)	General abuse protection without impacting POS/reception throughput

4. Authentication & Authorization

4.1 JWT Flow (S1 issues, S2–S4 verify)

1. Client `POST /s1/api/v1/auth/login` → S1 validates bcrypt hash + lock/active status.
2. S1 issues an **RS256**-signed access JWT (60 min TTL) + refresh token (30 days, SHA-256 hashed at rest, rotated on use).
3. Every request to S2–S4 carries `Authorization: Bearer <JWT>`.
4. The shared `JwtAuthenticate` middleware (identical pattern in S2, S3, S4) calls `POST {S1_URL}/api/v1/auth/verify` with header `X-Service-Key` + the bearer token.
5. S1 decodes/validates and returns `{valid, user{roles, permissions, name, employee_id}}`; the target system attaches `auth_user` to the request (per-request only, never persisted).
6. The `CheckPermission` middleware (`permission:S{n}.domain.resource.action`) enforces RBAC per route (Section 7).
7. On `401 TokenExpired`, the client calls `/s1/api/v1/auth/refresh` (rotates the refresh token).

4.2 JWT Payload (updated — adds `name`, ID2)

```
{
  "sub": "42",
  "employee_id": "17",
  "username": "abel.tadesse",
  "name": "Abel Tadesse",
  "roles": ["hr_manager", "report_viewer"],
  "permissions": ["S2.hr.employees.read", "S2.payroll.runs.read"],
  "dept_scope": "3",
  "iat": 1750000000,
  "exp": 1750003600,
  "iss": "wonderland-identity"
}
```

`name` is sourced from `users.display_name` in S1's database. **S1's `users` table gains a `display_name` VARCHAR(150) column** (Decision ID2), populated when S1 provisions the account in response to S2's `employee.created` / `employee.updated` events (Section 5.2), and kept in sync on subsequent updates. Any system that previously made a directory call purely to display “who is this person” (e.g. resolving a cashier's name for a receipt) now reads `name` straight from the verified JWT — **no cross-system call needed**.

4.3 Service-to-Service Auth

The shared `ServiceKeyAuthenticate` middleware (unchanged in spirit) protects the small number of remaining cross-system business endpoints:

```
// app/Http/Middleware/ServiceKeyAuthenticate.php – identical in S2, S3, S4
class ServiceKeyAuthenticate {
    public function handle($request, Closure $next) {
        if ($request->header('X-Service-Key') !== config('services.internal_key')) {
            abort(401, 'Invalid service key');
        }
        return $next($request);
    }
}
```



```
}  
}
```

Applied to (full list — see Section 6 for context of each):

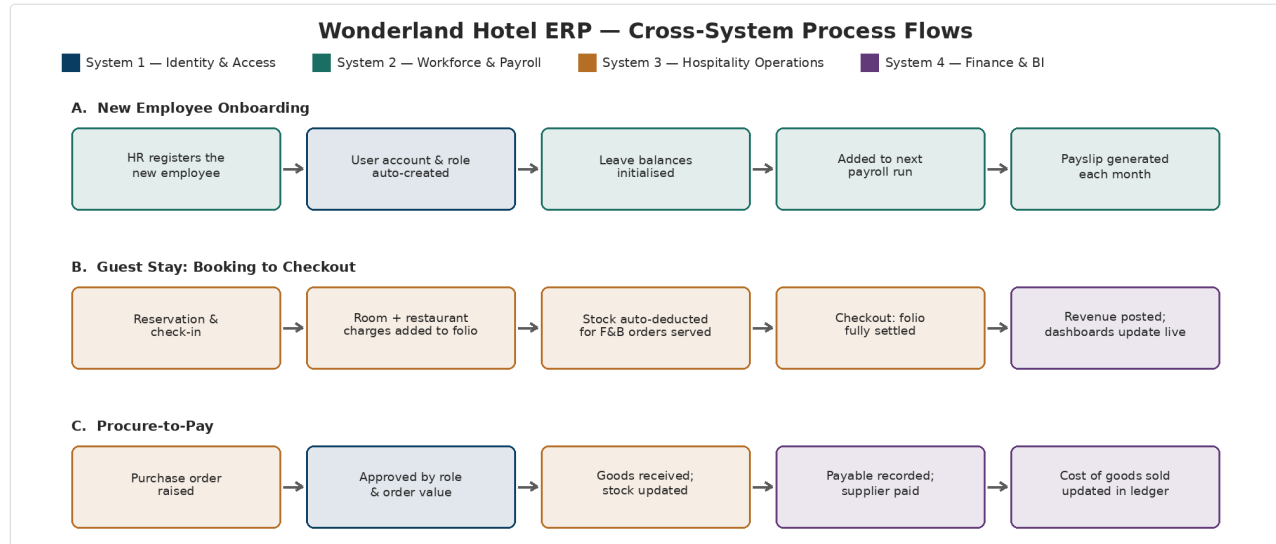
System	Endpoint	Caller
S2	<code>POST /api/v1/employees/{id}/deductions</code>	S3 (ID12)
S4	<code>POST /api/v1/journal-entries</code>	S2, S3
S1	<code>POST /api/v1/auth/verify</code>	S2, S3, S4

4.4 JWKS & Key Rotation

S1 publishes `GET /api/v1/auth/jwks` (unauthenticated) so its own RS256 signing keys can rotate via standard `kid`-based JWKS. **No other system needs to change** — S2–S4 never verify JWTs themselves; they always call S1's `/auth/verify`, so key rotation is entirely internal to S1.

5. Cross-System Integration Map

This is the authoritative list of **everything that crosses a system boundary**. If an interaction is not listed here, it is internal to one system and belongs in that system's own SDD.



Three representative end-to-end flows that span multiple systems: new-employee onboarding (S2 → S1 → S2), a guest stay from booking to checkout (entirely within S3, with revenue posted to S4 at the end), and procure-to-pay (S3 → S1 → S3 → S4).

5.1 Event Bus Topology

A shared Redis instance, `wh-redis-bus` (distinct from each system's own Horizon queue connection), is the pub/sub transport. Channel naming: `wh.events.{system}.{entity}.{action}` — e.g. `wh.events.s2.employee.created`. The PascalCase Laravel event class name (e.g. `EmployeeCreated`) remains the in-process name inside the emitting system; the subscriber re-dispatches its own local event of the same name on receipt.

Publishing goes through the transactional **outbox** pattern (Section 9, D2) — never a direct `PUBLISH` — so an event surviving the emitting system's DB commit is guaranteed to eventually publish even across a crash.

3 systems emit (S1, S2, S3); **S4 emits nothing** (ID8) — every dashboard/report S4 produces is either (a) native GL data already in S4's own database, or (b) data pulled on-demand from S1/S2/S3 with Redis TTL caching (Section 5.4), invalidated by the events below.

5.2 Cross-System Event Catalog

Channel	Emitted By	Consumed By	Payload	Effect
<code>wh.events.s2.employee.created</code>	S2	S1 (<code>CreateLinkedUserA</code> <code>employee_number,</code>	<code>employee_id,</code> <code>full_name,</code> <code>department_id,</code> <code>position_id,</code> <code>pension_category</code> <code>, basic_salary,</code> <code>default_role</code>	S1 creates <code>users</code> row (<code>must_change_password=true,</code> <code>display_name=full_name,</code> ID2), assigns default role for the employee's position
	S2			

Channel	Emitted By	Consumed By	Payload	Effect
wh.events.s2.employee.updated		S1 (SyncLinkedUserAccofull name,	employee_id, department_id, position_id	S1 updates users.display_name and department scope if changed (keeps JWT name / dept_scope accurate, ID2)
wh.events.s2.employee.archived	S2	S1 (DeactivateLinkedUseemployment_status — ID1)	employee_id, s: "archived", archived_at, reason	S1 sets the linked users row is_active=false, force-revokes refresh tokens, writes an audit-log entry
wh.events.s1.permission.changed	S1	S4 (cache invalidation)	role_id, permission_ids: [], action: "sync" "grant" "revoke"	Refreshes S4's admin/RBAC dashboard cache
wh.events.s2.payroll_run.approved	S2	S4 (cache invalidation)	payroll_run_id, period, total_net_pay, employee_count	Refreshes payroll dashboards/reports (journal posting itself is a separate sync call, Section 5.3)
wh.events.s2.severance.calculated	S2	S4 (cache invalidation)	employee_id, severance_amount, years_of_service, unused_leave_days, net_of_loans_and_deductions	Refreshes severance/loans report
wh.events.s2.leave.approved	S2	S4 (cache invalidation)	employee_id, leave_request_id, leave_type, start_date, end_date, days	Refreshes leave-utilization dashboard
wh.events.s3.goods.received	S3	S4 (cache invalidation)	goods_receipt_id, purchase_order_id, supplier_id, total_value, item_count	Refreshes inventory/procurement dashboards (AP journal is a separate sync call)
wh.events.s3.purchase_order.approved	S3	S4 (cache invalidation)	purchase_order_id, supplier_id, total_amount, approval_tier: 1 2 3	Refreshes procurement dashboard
wh.events.s3.order.finalized	S3	S4 (cache invalidation)	order_id, customer_type, customer_ref_id, subtotal,	Refreshes F&B sales dashboard

Channel	Emitted By	Consumed By	Payload	Effect
			<code>service_charge,</code> <code>vat,</code> <code>total</code>	
<code>wh.events.s3.employee_consumption_period.closed</code>	S3	S4 (cache invalidation)	<code>period,</code> <code>employee_count,</code> <code>total_amount</code>	Refreshes staff-consumption report
<code>wh.events.s3.guest.checked_in</code>	S3	S4 (cache invalidation)	<code>reservation_id,</code> <code>room_id,</code> <code>folio_id,</code> <code>check_in_at</code>	Refreshes real-time occupancy dashboard
<code>wh.events.s3.guest.checked_out</code>	S3	S4 (cache invalidation)	<code>reservation_id,</code> <code>room_id,</code> <code>folio_id,</code> <code>check_out_at,</code> <code>total_charges</code>	Refreshes occupancy/revenue dashboard
<code>wh.events.s3.folio.settled</code>	S3	S4 (cache invalidation)	<code>folio_id,</code> <code>room_id,</code> <code>reservation_id,</code> <code>total_charges,</code> <code>total_paid,</code> <code>service_charge,</code> <code>vat,</code> <code>settled_at</code>	Refreshes revenue dashboard (room-revenue journal is a separate sync call)

13 channels total, but only **3 emitting systems** (was 8 emitting modules) and **1 consumer for the dashboard-invalidation channels (S4)** — each system runs exactly one `redis-bus-subscriber` Horizon worker.

5.3 Cross-System Synchronous Calls

Caller	Callee	Endpoint	Purpose	Idempotent?
S2, S3, S4 (every request)	S1	<code>POST /api/v1/auth/verify</code>	JWT validation	n/a (read-only)
S2	S4	<code>POST /api/v1/journal-entries</code>	Payroll-run journal, severance journal, loan disbursement/repayment journals	Yes (<code>Idempotency-Key</code> , Section 9 D5)
S3	S4	<code>POST /api/v1/journal-entries</code>	AP/goods-receipt + COGS-on-dispatch journal, daily F&B sales summary (netted per ID3), room-revenue/folio-settlement journal, event AR journal	Yes
S3	S2	<code>POST /api/v1/employees/{id}/deductions</code> (ID12)	Push staff F&B consumption deduction for next payroll run	Yes
S4	S1	<code>GET /api/v1/users</code> , <code>/roles</code> , <code>/audit-logs</code>	Admin/security dashboards & reports	n/a (read-only, cached)

Caller	Callee	Endpoint	Purpose	Idempotent?
S4	S2	<code>GET /api/v1/employees, /payroll-runs, /leave-requests, /attendance-records</code>	HR/payroll/leave dashboards & reports	n/a (read-only, cached)
S4	S3	<code>GET /api/v1/rooms, /reservations, /orders, /items, /purchase-orders</code>	Occupancy/F&B/inventory dashboards & reports	n/a (read-only, cached)

This is the **entire** cross-system surface. Everything else — every interaction that was previously `M2↔M3`, `M3↔M4`, `M5↔M6`, `M6↔M7`, `M8↔M9` — is now internal to one system (Section 5.5).

5.4 S4's Read-Through Cache (unchanged pattern, fewer upstreams)

S4's Reporting & BI module retains M9's original “owns no primary data outside the GL” design: dashboards needing occupancy, F&B sales, HR/payroll, leave, or inventory data call S1/S2/S3's read APIs and cache the result in Redis with per-data-type TTLs, invalidated early by the events in 5.2:

Data type	TTL	Source System
Live revenue / occupancy / F&B sales	60s	S3
Inventory levels	120s	S3
HR / payroll / leave summaries	300s	S2
Financial reports (Trial Balance, P&L, etc.)	900s (or none — native S4 data)	S4 (native)
RTM / UAT tracker	uncached	S4 (native)

5.5 What Became Internal (the actual simplification)

Former Cross-Service Interaction	Now	Decision
<code>M2 → M4 : EmployeeCreated</code> initialises leave balances	In-process: HR module fires a local event, Leave module's observer creates <code>leave_balances</code> rows in the same transaction	—
<code>M2 → M4 : EmployeeArchived</code> closes open leave requests	In-process observer	—
<code>M2 ↔ M3</code> : severance calc reads employee tenure/pension data; Dead File update after <code>SeveranceCalculated</code>	In-process: <code>SeveranceService</code> queries <code>employees</code> directly; writes <code>offboarding_records.severance_amount</code> in the same transaction	—
<code>M3 → M2 : GET /employees</code> pull at payroll-run start	Direct Eloquent query against <code>employees</code>	ID6
		ID6

Former Cross-Service Interaction	Now	Decision
M3 → M4 : <code>unused-leave-balance</code> , <code>overtime-approved</code>	Direct Eloquent query against <code>leave_balances</code> / <code>overtime_records</code>	
M6 → M5 : <code>POST /items/dispatch</code> (ingredient deduction)	Single in-process transaction calling <code>FifoFefoService::dispatch()</code>	ID4
M6 → M7 : <code>GET /active-folio</code> → <code>POST /fmb-charge</code>	Single in-process call to <code>FolioService::postChargeForRoom()</code>	ID5
M6 & M7 separate GL postings + “wash account” netting (Decision C2)	S3 nets internally, sends one journal to S4	ID3
M8 → M9 : <code>JournalPosted</code> cache invalidation	In-process Eloquent observer on <code>JournalEntry</code>	ID7
M9 querying M8’s GL tables over HTTP	Direct query against <code>journal_entries</code> / <code>journal_lines</code> / <code>accounts</code> in the same database	—

6. Internal Module Maps (summary — full detail in each system's SDD)

System	Internal Modules	Shared Database Tables Across Modules?
S2	HR (employees, departments, positions, disciplinary, assets, guarantors, offboarding) · Payroll (payroll runs/entries/earnings/deductions, overtime rates, loans) · Leave & Attendance (leave types/balances/requests, overtime records, attendance)	Yes — Payroll and Leave both FK directly to <code>employees</code> ; this is the main benefit of the merge (no more cross-DB joins for the most data-coupled relationship in the whole ERP)
S3	Inventory & Purchasing (items, stock batches/movements, POs, suppliers, goods receipts, supplier payments) · Restaurant & F&B (menu items, recipes, orders, bills, events) · Hotel & Room Mgmt (rooms, reservations, folios, charges, payments, group bookings)	Yes — Restaurant's <code>recipe_ingredients</code> FK directly to Inventory's <code>items</code> ; Restaurant's order-finalization writes directly to Hotel's <code>folio_charges</code>
S4	GL & Accounting (accounts, journal entries/lines, AR/AP) · Reporting & Analytics (RTM/UAT trackers, dashboard cache config)	Reporting queries GL tables directly; no duplicated data
S1	Identity & Access (users, roles, permissions, audit logs, refresh tokens)	Single module, no internal split needed

7. RBAC — Roles, Systems, and the Permission Convention

7.1 Permission Naming Convention (ID9)

`S{n}.{domain}.{resource}.{action}` — e.g. `S2.payroll.runs.execute`, `S2.hr.employees.read`, `S3.hotel.folios.read`, `S3.inventory.purchase_orders.approve`, `S4.gl.journal_entries.post`, `S1.identity.audit_logs.read`.

Each system SDD (S1–S4) carries the **full permission catalog** for its own domains (every endpoint × permission string × default role). This document carries only the **role** → **system access** matrix below, since several roles span systems.

7.2 Role → System Access Matrix (12 roles, unchanged set from the macro design)

Role	S1	S2	S3	S4
super_admin	Full	Full	Full	Full
general_manager	Read	Read	PO approval (tier 3) + read	Full read + top-tier journal approval
finance_manager	—	—	PO approval (tier 2)	Full (journals, AR/AP)
accountant	—	—	—	Full (GL, journals, reports)
hr_manager	—	Full (HR + Leave)	—	—
payroll_officer	—	Full (Payroll)	—	—
department_head	—	Leave approval + dept read	PO approval (tier 1)	—
inventory_manager	—	—	Full (Inventory)	—
restaurant_manager	—	—	Full (Restaurant & F&B)	—
receptionist	—	—	Full (Hotel & Rooms)	—
cashier	—	—	Transactions (Restaurant + Hotel)	—
report_viewer	Read	Read	Read	Read (dashboards/ reports)

Observation: most roles now touch **one, or at most two**, systems — down from spanning up to four modules. `department_head` and `finance_manager` / `general_manager` are the only roles that genuinely need to reason about more than one system, and both cases map cleanly onto the **approval-chain** relationship between S2/S3 (where the spend/leave originates) and S4 (where it's recorded).

8. Financial Posting Map (S2/S3 → S4)

The chart of accounts (24 codes) lives in S4 and is unchanged in content from the original M8 design — only the **posting source** changes from module-level to system-level.

Account Code	Account	Posted By
1001 Cash	S3 (cash F&B sales, cash folio payments, cash supplier payments)	
1002 Bank (CBE)	S3 (bank-transfer payments, both directions)	
1003 Mobile Banking Clearing	S3 (mobile-banking payments)	
1004 POS Clearing	S3 (POS payments)	
1005 VISA Clearing	S3 (VISA payments, supplier + folio)	
1100/1101 AR (Guests/Events)	S4 receivable_records (from S3 event/guest charges)	
1102 Staff Loans Receivable	S2 (loan disbursement debits; repayment via payroll deduction credits)	
1200/1201 Inventory	S3 (goods receipt debits; ingredient dispatch credits for COGS)	
2001 AP — Suppliers	S3 (goods receipt / supplier payments)	
2100–2200 Payroll liabilities	S2 (payroll-run approval)	
2300 VAT Payable	S3 (F&B sales VAT + folio VAT, single netted entry per ID3)	
4001 Room Revenue	S3 (FolioSettled)	
4002 F&B Revenue	S3 (daily F&B sales summary, all customer types in one netted entry — ID3 removes the old per-type “wash account” split)	
4003 Service Charge Income	S3 (folio + F&B, single netted entry per ID3)	
4004 Event Revenue	S3 (event charges)	
5001–5004 Expenses (Salaries, Pension, Allowances, Overtime)	S2 (payroll-run approval)	
5003 Cost of Food Sold	S3 (ingredient dispatch → COGS, valued at FIFO/FEFO batch cost, ID4)	
5005 Severance Expense	S2 (<code>severance.calculated</code> → severance payout journal)	

8.1 The ID3 Simplification in Detail

Previously (9-module design), M6 (Restaurant) posted a daily F&B sales summary covering **all 7 customer types**, but for `hotel_guest` orders this double-counted against M7's (Hotel) `FolioSettled` posting — solved with a “wash account” (`4002.1 F&B Revenue – Room-Billed`) that S4's reporting layer had to net out.

Now that Restaurant and Hotel are **both inside S3**, S3 computes the correct net figures *before* calling S4:

S3's daily F&B summary journal excludes hotel_guest orders entirely (their revenue/SC/VAT is already included in the FolioSettled journal that S3 sends when each such folio is settled)

Dr 1001/1003/1004/1005 (sum of non-hotel-guest payment methods)
 Dr 1100 AR – Event (event-type orders)
 Cr 4002 F&B Revenue (non-hotel-guest subtotals)
 Cr 4003 Service Charge Income
 Cr 2300 VAT Payable

S4 receives two clean, non-overlapping journals (room-revenue-with-FnB from FolioSettled, and FnB-sales-summary-excluding-room-billed) and needs **no departmental-allocation convention** in its reporting layer. This is a direct consequence of the modular-monolith merge and should be implemented in S3's order-finalization / folio-settlement services (full detail in [S3's SDD](#)).

8.2 Severance & Loans (unchanged from original design)

Severance payout:	Dr 5005 Severance Expense / Cr 1001 Cash (or 1002 Bank)
Loan disbursement:	Dr 1102 Staff Loans Receivable / Cr 1001 Cash (or 1002 Bank)
Loan repayment (payroll):	Dr 2100 Payroll Liability / Cr 1102 Staff Loans Receivable

9. Platform Conventions

These apply uniformly to all four systems. (Adapted from the original Appendix D, retargeted from 9 modules to 4 systems.)

#	Topic	Convention
D1	Event transport	<code>wh-redis-bus</code> pub/sub, channels <code>wh.events.{system}.{entity}.{action}</code> (Section 5.1–5.2)
D2	Event reliability	Transactional <code>event_outbox</code> table in S1, S2, S3 (not S4 — emits nothing). <code>outbox:publish</code> job every 10s: <code>pending</code> → <code>PUBLISH</code> → <code>published</code> ; 5 retries with backoff [<code>10s</code> , <code>60s</code> , <code>5m</code> , <code>30m</code> , <code>2h</code>], then <code>failed</code>
D3	Event envelope	<code>{"event": "wh.events.s3.folio.settled", "version": 1, "occurred_at": "...", "source_system": "S3", "request_id": "...", "payload": {...}}.version</code> allows additive evolution; consumers ignore unknown fields
D4	Retry / dead-letter	Subscriber jobs: Horizon default 3 attempts, backoff [<code>10s</code> , <code>60s</code> , <code>5m</code>], then <code>failed_jobs</code> . Any <code>event_outbox</code> row reaching <code>status=failed</code> , or any subscriber job in <code>failed_jobs</code> , triggers a local <code>NotificationService::send()</code> to <code>super_admin</code>
D5	Payment idempotency	<code>Idempotency-Key</code> header (UUID v4) + <code>idempotency_keys</code> table in S2, S3, S4 . Applies to: S2 <code>POST /payroll-runs/{id}/approve</code> , <code>POST /employees/{id}/loans</code> ; S3 <code>POST /supplier-payments</code> , <code>PUT /purchase-orders/{id}/approve</code> , <code>POST /bills/{id}/payments</code> , <code>POST /folios/{id}/payments</code> , <code>POST /employees/{id}/deductions</code> (outbound to S2); S4 <code>POST /journal-entries</code> . Same-key+same-body replays cached response; same-key+different-body → <code>409 IDEMPOTENCY_KEY_CONFLICT</code>
D6	Pagination	Every <code>GET</code> collection endpoint: <code>?page=&per_page=</code> (default 25, max 100) → <code>{"data":[...], "meta":{"current_page","per_page","total","last_page"},"links":</code>

#	Topic	Convention
		<code>{"first", "last", "prev", "next" }</code>
D7	Error format	<pre> {"error": {"code", "message", "details"} }. Standard codes: VALIDATION_ERROR (400), UNAUTHENTICATED / INVALID_SERVICE_KEY (401), FORBIDDEN (403), NOT_FOUND (404), IDEMPOTENCY_KEY_CONFLICT / UNBALANCED_JOURNAL / INSUFFICIENT_STOCK / INVALID_STATE_TRANSITION (409), UNPROCESSABLE (422), RATE_LIMITED (429), INTERNAL_ERROR (500), SERVICE_UNAVAILABLE (503) </pre>
D8	Secrets & rotation	Prod secrets via Docker <code>_FILE</code> convention. <code>ServiceKeyAuthenticate</code> accepts <code>INTERNAL_KEY_CURRENT</code> and <code>INTERNAL_KEY_PREVIOUS</code> for rotation (deploy new key to all 4 systems as <code>_PREVIOUS</code>, then <code>_CURRENT</code>, remove <code>_PREVIOUS</code> after 24h). JWT signing-key rotation is internal to S1 (<code>GET /api/v1/auth/jwks</code>)
D9	Logging & correlation	JSON stdout logs → Loki/Promtail. <code>wh-gateway</code> generates <code>X-Request-Id</code> if absent, propagated to all downstream/sync calls and embedded in every bus envelope (D3)
D10	Tracing	Deferred; <code>X-Request-Id</code> (D9) is the foundation for adding OpenTelemetry later
D11	CI/CD	Per-system GitHub Actions: test (PHPUnit + PHPStan against MySQL 8 + Redis 7 services) → build → push (tag = git SHA) → manual promotion to staging/prod via GitHub Environments. Blue-green/k8s explicitly out of scope
D12	Testing strategy	Feature tests per system; contract tests via <code>GET /api/v1/openapi.json</code> validated against the calls in Section 5.3; event-integration tests (<code>docker-compose.test.yml</code> pairs: S2 ↔ S1, S3 ↔ S2, S2 ↔ S4, S3 ↔ S4, S1 ↔ S4) covering Section 5.2 end-to-end; UAT via S4's RTM/UAT tracker (50/50 FR codes)

#	Topic	Convention
D13	Soft delete	No hard <code>DELETE</code> anywhere — status/flag columns only (<code>is_active</code> , <code>employment_status</code> , <code>deactivated_at</code> , etc.), generalizing S2's Dead File pattern
D14	Backup / DR	Nightly encrypted <code>mysqldump</code> per database (4, not 9), 30 days local + 90 days offsite; <code>innodb_encrypt_tables=ON</code> for all 4; quarterly DR restore drill logged to S1's <code>audit_logs</code>

New tables (summary): `event_outbox` — S1, S2, S3. `idempotency_keys` — S2, S3, S4. **New jobs (summary):** `outbox:publish` (S1, S2, S3, every 10s). `idempotency:purge` (S2, S3, S4, daily 02:00). **New endpoint:** `GET /api/v1/auth/jwks` (S1). **New middleware:** `EnsureIdempotency` (S2, S3, S4). `redis-bus-subscriber` Horizon worker (S1, S2, S3, S4 — S4 subscribes only, per ID8).

10. Deployment Topology

```

services:
  wh-gateway:
    image: nginx:1.27
    ports: ["80:80", "443:443"]
    volumes:
      - ./gateway/nginx.conf:/etc/nginx/nginx.conf:ro
      - ./gateway/certs:/etc/nginx/certs:ro
    depends_on: [s1-identity, s2-workforce, s3-hospitality, s4-finance-bi]
    networks: [wh-internal]

  s1-identity:
    build: ./s1-identity-access
    environment:
      - DB_DATABASE=wh_s1_db
      - REDIS_BUS_HOST=wh-redis-bus
    networks: [wh-internal]
    # no published ports – reachable only via wh-gateway or wh-internal

  s2-workforce:
    build: ./s2-workforce-payroll
    environment:
      - DB_DATABASE=wh_s2_db
      - IDENTITY_SERVICE_URL=http://s1-identity:9001
      - REDIS_BUS_HOST=wh-redis-bus
    networks: [wh-internal]

  s3-hospitality:
    build: ./s3-hospitality-operations
    environment:
      - DB_DATABASE=wh_s3_db
      - IDENTITY_SERVICE_URL=http://s1-identity:9001
      - WORKFORCE_SERVICE_URL=http://s2-workforce:9002
      - REDIS_BUS_HOST=wh-redis-bus
    networks: [wh-internal]

  s4-finance-bi:
    build: ./s4-finance-bi
    environment:
      - DB_DATABASE=wh_s4_db
      - IDENTITY_SERVICE_URL=http://s1-identity:9001
      - WORKFORCE_SERVICE_URL=http://s2-workforce:9002
      - HOSPITALITY_SERVICE_URL=http://s3-hospitality:9003
      - REDIS_BUS_HOST=wh-redis-bus
    networks: [wh-internal]

  wh-redis-bus:
    image: redis:7
    networks: [wh-internal]

  wh-mysql:
    image: mysql:8.0
    environment:
      MYSQL_DATABASE: wh_s1_db # wh_s2_db..wh_s4_db created via init scripts
    volumes: ["/mysql-init:/docker-entrypoint-initdb.d:ro"]
    networks: [wh-internal]

networks:

```

```
wh-internal:  
  driver: bridge
```

Four application containers (down from nine), one shared MySQL instance hosting four databases (`wh_s1_db`...
`wh_s4_db`), one shared event-bus Redis, plus each system's own Horizon queue Redis connection (omitted
above for brevity — same per-system `.env` pattern as the original design).

11. Decision Register (Full Index)

ID	One-line summary	Detail
ID1	<code>EmployeeArchived</code> → S1 deactivates the linked user account (new)	§5.2
ID2	JWT gains <code>name</code> claim; S1 <code>users.display_name</code> (new)	§4.2
ID3	S3 nets hotel-guest F&B internally before posting to S4 — no wash account	§8.1
ID4	Ingredient dispatch is an in-process S3 transaction	§5.5
ID5	Active-folio + F&B charge posting is an in-process S3 call	§5.5
ID6	Payroll reads HR/Leave data directly (no pull-based framing)	§5.5
ID7	<code>JournalPosted</code> → dashboard refresh is an in-process S4 observer	§5.5
ID8	Bus channels renamed to system-level; S4 emits nothing	§5.1–5.2
ID9	Permission convention: <code>S{n}.</code> <code>{domain}.{resource}.{action}</code>	§7.1
ID10	<code>event_outbox</code> in S1/S2/S3 only; <code>idempotency_keys</code> in S2/S3/S4 only	§9
ID11	Gateway: 4 upstreams	§3
ID12	S3 → S2 food-consumption push remains the one cross-system business call between S2/S3	§5.3

12. Glossary & Legacy Cross-Reference

Term	Meaning
System (S1–S4)	One of the four modular-monolith Laravel applications described in the macro architecture document
Internal Module	A former M1–M9 microservice, now a namespace/module inside one of the four systems, sharing its database
Bus	<code>wh-redis-bus</code> , the cross-system pub/sub event transport (Section 5.1)
Outbox	<code>event_outbox</code> table — transactional guarantee that DB-committed events are eventually published
Directory claim	The <code>name</code> / <code>employee_id</code> fields in the JWT, used in place of cross-system lookups (ID2)
Netted journal	A GL entry that S3 has already de-duplicated/combined internally before sending to S4 (ID3)

For the **original, pre-consolidation** module-by-module detail (useful for understanding *why* a given table/endpoint/rule exists, or for historical traceability), see [00_MASTER_Wonderland_ERP_Design_Documentation.md](#) and [M1_*.md](#) — [M9_*.md](#). Every table, endpoint, and business rule in S1–S4's SDDs traces back to a specific module doc; S1–S4's SDDs are the documents developers should build from, but the legacy docs remain the system of record for *why* a rule exists (FR traceability, Appendix B).

End of Document — S0, Cross-System Integration & Platform Conventions, v1.0